

Conflux Rust

v3.1.0

Code Security Audit

Prepared for Conflux Network

Report Version 1.1 | August 10, 2026 | Final Report

Audit Overview

Project	Conflux Rust
Prepared for	Conflux Network
Audit Team	Conflux Team
Assessment Type	Code Security Audit
Repository	Conflux-Chain/conflux-rust
Version	v3.1.0
Assessed Commit	6e2e837c8271695b139e491f50e586ebd8ed10c8
Report Version	v1.1
Report Date	2026-08-10

Results Summary

Critical	High	Medium	Low	Total	Open	Reported Fixed
0	1	2	7	10	6	4

The audit confirmed 10 security findings. Four are marked as remediated in project records and six remain Open. No unresolved Critical or High-severity finding was identified in the target version. All currently open findings are Low severity and primarily concern public RPC resource boundaries, historical EVM rules, and risks dependent on compatibility or deployment conditions.

Contents

01	Executive Summary
02	Audit Scope
03	Audit Methodology
04	Risk Methodology and Status Definitions
05	Findings Summary
06	Detailed Findings
07	Remediation Priorities
08	Limitations and Disclaimer
A	Code Locations and Remediation References

1. Executive Summary

1.1 Assessment Overview

This report presents a code security audit of Conflux Rust v3.1.0 at Git revision 6e2e837c8271695b139e491f50e586ebd8ed10c8. The audit considered the target revision as a whole and applied a risk-oriented, systematic code review methodology across security-relevant areas including consensus-critical paths, EVM execution, transaction processing, P2P synchronization, the PoW/PoS event bridge, RPC services, and node configuration. The 10 findings documented in this report were identified and confirmed through that repository-wide review.

1.2 Assessment Conclusion

The audit confirmed 10 security findings: 0 Critical, 1 High, 2 Medium, and 7 Low. As of this report version, project records indicate that the High finding, both Medium findings, and one Low finding have been remediated; the remaining six Low findings should be further addressed, formally accepted, or documented. Where project records indicate a fix but this report preparation did not independently re-run the proof of concept, mainnet RPC testing, or patch regression testing, the status is recorded as Reported Fixed.

Within the target revision and the audit scope defined in this report, no unresolved Critical or High-severity finding was identified. The remaining open issues primarily concern historical EVM rule boundaries, resource limits and secure defaults for public RPC services, specific transaction compatibility policies, and edge behavior in private chains, development networks, or historical replay environments. This conclusion applies only to the code revision and scope defined in this report. As with any code security audit, the objective is to identify and reduce discoverable risk; it does not constitute an absolute guarantee that the software contains no unknown defects.

1.3 Risk Statistics

Severity	Total	Open	Reported Fixed
Critical	0	0	0
High	1	0	1
Medium	2	0	2
Low	7	6	1
Total	10	6	4

2. Audit Scope

2.1 Target Code and Revision

Repository	Conflux-Chain/conflux-rust
Version	v3.1.0
Commit	6e2e837c8271695b139e491f50e586ebd8ed10c8

For hardfork-related findings, reachability was assessed against the activation boundaries and mainnet trigger conditions applicable to the target revision. For issues with existing remediation records, the associated pull request and project-reported remediation status are recorded in the relevant finding.

2.2 Primary Code Domains Reviewed

The audit considered the full codebase at the target commit. To make the security boundaries clear, the table below summarizes the principal security domains examined, the primary review objectives, and representative source locations. These paths illustrate the areas of emphasis and do not further narrow the repository-wide audit scope.

Code Domain	Primary Review Focus	Representative Source Locations
EVM Execution and Protocol Semantics	Opcode behavior, memory semantics, hardfork/CIP gating, exceptional paths, and canonical execution.	<code>crates/execution/{executor,vm- interpreter,vm-types,solidity- abi}</code>
Consensus and State Bridging	PoW/PoS data bridging, pivot/canonical identity, and consistency between execution results and event caches.	<code>crates/cfxcore/core/src/ {consensus,pos}</code>
P2P and Synchronization	Untrusted peer messages, request/response binding, length and index relationships, and malformed response handling.	<code>crates/cfxcore/core/src/sync</code>
JSON-RPC and Resource Controls	eth/trace RPC, log queries, batching, throttling, concurrency, and expensive request boundaries.	<code>crates/rpc; crates/client/src/rpc_starter.rs</code>
Transaction Parsing and Validation	eSpace/Ethereum legacy transactions, chain ID handling, replay protection, and transaction validity.	<code>crates/primitives/src/ transaction; crates/cfxcore/core/src/verificat ion.rs</code>
Node Configuration and CLI	CLI argument mapping, runtime configuration consumption, secure defaults, and service exposure.	<code>bins/conflux/src/cli.rs; crates/config; run/*.toml</code>

2.3 Core Security Considerations

The review was organized around several core security invariants. Attacker-controlled input should not trigger panics in consensus or other critical worker paths. EVM behavior should remain consistent with the activation state of the relevant EIPs and CIPs. Fork, pivot, and cache relationships should preserve canonical identity. Network responses should be validated for length, type, and request-context consistency before use. Public RPC services should enforce appropriate limits on range, result count, concurrency, batching, timeouts, and request-body size. CLI and runtime configuration mappings should be explicit and testable, and configuration failures should not be silently ignored in a way that produces unintended runtime behavior.

2.4 Out-of-Scope Items

The primary audit target was the `conflux-rust` code at the specified commit. The engagement did not include a complete supply-chain audit of third-party dependencies or upstream projects, nor did it assess the security of node host systems, cloud platforms, perimeter devices, or other external infrastructure. No independent destructive testing was performed against production environments, and not every remediation patch was re-tested dynamically. Code introduced or modified after the target commit, as well as operational processes, key management, and organizational controls outside the project codebase, are not fully covered by this report.

3. Audit Methodology

The audit assessed the `conflux-rust` repository at the target commit, with risk-oriented manual code review as the primary technique, supplemented by architecture analysis, attack-surface analysis, and security-invariant checks. The review began by examining data flows and trust relationships between node modules, identifying low-trust inputs originating from transactions, contracts, RPC clients, P2P peers, on-chain state, and local configuration, and tracing those inputs into consensus, execution, and privileged paths to evaluate boundary validation and error handling.

At the consensus and execution layers, the review focused on EVM execution, transaction validation, canonical execution, PoW/PoS data bridging, and hardfork/CIP gating. Particular attention was given to panic conditions, inconsistent state transitions, activation-rule errors, canonical-identity violations, and cross-module data contamination. For P2P and synchronization logic, the audit evaluated whether messages and responses from untrusted peers remain correctly bound to the originating request context, whether length, index, and type relationships are validated before use, and whether malformed network input can be transformed into out-of-bounds access, unhandled failures, or node-level denial of service.

At external service and operational boundaries, the audit examined parameter ranges, query breadth, batch execution, concurrency, throttling, timeout behavior, and expensive execution paths exposed through public JSON-RPC. The objective was to assess whether low-trust clients could amplify CPU, memory, storage I/O, or backend execution cost. The review also examined CLI-to-runtime configuration mapping and secure defaults, with emphasis on silently ineffective configuration, unintended service exposure, and resource controls that are disabled by default. EIP/CIP behavior, historical rules, and hardfork activation conditions were evaluated against reachability on current mainnet, historical windows, private chains, and development networks.

Potential issues identified during review were further assessed for trigger conditions, attacker prerequisites, impact scope, and mainnet reachability, and were then rated using the severity model defined in this report. The 10 final findings are the result of this repository-wide audit process. Where the project provided remediation records, this report records the associated pull request and remediation status; where independent dynamic regression testing was not repeated during this engagement, the status is Reported Fixed.

3.1 Audit Phases and Review Focus

Phase	Review Focus
Architecture and Threat Modeling	Identify trust boundaries across transactions, P2P, RPC, on-chain state, and local configuration; establish critical assets, entry points, and security invariants.
Systematic Manual Code Review	Trace low-trust input through consensus, EVM, synchronization, RPC, transaction validation, and configuration code; examine logic errors, panics, boundary conditions, and state consistency.
Protocol and Compatibility Review	Evaluate EIP/CIP behavior, hardfork activation boundaries, mainnet reachability, historical semantics, and cross-implementation compatibility risk.
Targeted Technical Review	For identified issues, assess trigger paths, attacker prerequisites, impact scope, mainnet reachability, and remediation records to determine rating and remediation priority.

The phases above were driven primarily by manual analysis. This report does not claim automated scanning, fuzzing, or production attack simulation that was not actually performed. Existing external records or project

remediation information were used only as supporting evidence where they could be directly traced to a specific finding.

4. Risk Methodology and Status Definitions

4.1 Impact and Likelihood

This report rates findings using an Impact x Likelihood model. Impact describes the consequences of successful exploitation or triggering of a finding, including effects on consensus integrity, consensus or node availability, execution correctness, public RPC services, and operational security. Likelihood describes the practical probability that an attacker can satisfy the trigger conditions and cause the stated impact in the target revision under realistic deployment conditions. Mainnet reachability and deployment prerequisites directly influence Likelihood for historical hardfork paths, private-chain-only paths, or issues exposed only by specific operator configurations.

Level	Impact Definition
High	May compromise consensus integrity, persistently block canonical execution, cause remote process-level denial of service, create major state errors, or produce equivalent protocol-security impact.
Medium	May significantly reduce single-node availability, contaminate canonical data, exhaust public RPC resources, cause important protocol behavior errors, or create material compatibility or operational risk.
Low	Impact is primarily limited to specific historical rules, private/development networks, localized compatibility, configuration reliability, or constrained operational scenarios and does not directly create a major current-mainnet security consequence.

Level	Likelihood Definition
High	Directly triggerable on current mainnet or common deployments by a remote or low-privilege attacker with few prerequisites and realistic repeatability.
Medium	Requires a specific protocol state, request/response relationship, service exposure, or a limited set of satisfiable prerequisites.
Low	Not reachable on current mainnet, limited to historical rules or private/development networks, or dependent on narrow account state, operator configuration, or local operational conditions.

4.2 Severity Matrix

Final severity is derived from the combination of Impact and Likelihood. The matrix is intended to keep ratings consistent and to avoid over-rating issues whose theoretical maximum impact is high but whose practical reachability is limited.

Impact / Likelihood	Low	Medium	High
High	Medium	High	Critical
Medium	Low	Medium	High
Low	Low	Low	Medium

4.3 Finding Status

Status	Definition
Open	The issue remains present, or remediation has not been confirmed.
Reported Fixed	Project records or an associated pull request indicate remediation, but the patch was not independently re-validated through dynamic regression testing during this assessment.
Acknowledged	The project accepts the risk or explicitly retains the behavior as a design, compatibility, or operational policy.
Not Applicable	Review confirms that the issue does not apply to the target revision or target deployment environment.

5. Findings Summary

5.1 Risk Distribution

The table below maps all 10 findings onto the Impact x Likelihood matrix. It highlights where practical risk is concentrated in the target version: the single High finding is assessed as High Impact / Medium Likelihood; the two Medium findings are assessed as High Impact / Low Likelihood and Medium Impact / Medium Likelihood; the remaining findings fall into Low severity because they are not reachable on current mainnet, depend on limited deployment conditions, or have a narrower impact boundary.

Impact / Likelihood	Low	Medium	High
High	CFX-002	CFX-001	-
Medium	CFX-004, CFX-005 CFX-006, CFX-007 CFX-009	CFX-003	-
Low	CFX-010	CFX-008	-

5.2 Findings Overview

ID	Finding	Severity	Impact	Likelihood	Status
CFX-001	Byte-based truncation of UTF-8 revert reason can panic canonical execution	High	High	Medium	Reported Fixed
CFX-002	Non-pivot staking events can contaminate the PoS event cache	Medium	High	Low	Reported Fixed
CFX-003	Unchecked GetTransactionsResponse hash count can cause an out-of-bounds panic	Medium	Medium	Medium	Reported Fixed
CFX-004	BLOCKHASH(U256::MAX) can overflow without a guard	Low	Medium	Low	Open
CFX-005	pre-CIP-645 BLOBHASH/BLOBB ASEFEE metadata gap can panic	Low	Medium	Low	Open
CFX-006	eth_getLogs limits are disabled by default	Low	Medium	Low	Open

ID	Finding	Severity	Impact	Likelihood	Status
CFX-007	eth_* / trace_* RPC methods bypass throttling coverage	Low	Medium	Low	Open
CFX-008	MCOPY lacks zero-fill for fully out-of-range source memory under legacy rules	Low	Low	Medium	Open
CFX-009	eSpace accepts unprotected pre-EIP-155 transactions	Low	Medium	Low	Open
CFX-010	CLI argument IDs can mismatch configuration fields and silently fail	Low	Low	Low	Reported Fixed

6. Detailed Findings

CFX-001 Byte-based truncation of UTF-8 revert reason can panic canonical execution

Severity	Impact	Likelihood	Status
High	High	Medium	Reported Fixed

Affected Component	Solidity revert-reason decoding and canonical execution-result construction
Attack Surface	Attacker-controlled EVM revert data
Mainnet Reachability	Reachable on the canonical execution path before remediation
Code Location	crates/execution/solidity-abi/src/utls.rs :: string_revert_reason_decode()
Remediation Reference	Project records mark the issue fixed; no PR number was provided

Description. After validating a Solidity Error(string) revert reason as UTF-8, the code truncates the string using the fixed byte slice `str[0..50]`. If byte 50 falls inside a multibyte UTF-8 character, Rust string slicing panics because the index is not on a valid character boundary. The logic is used while constructing canonical block execution results, so a crafted revert reason can convert an otherwise ordinary failed EVM transaction into a panic on the execution path.

Security Impact. The finding can prevent normal construction of transaction execution results, receipts, and the epoch execution commitment, potentially blocking a critical execution path and affecting consensus availability. An attacker only needs to submit a transaction or contract call that produces a specifically crafted multibyte UTF-8 revert reason.

Root Cause. Attacker-controlled UTF-8 text is sliced at a fixed byte offset without first ensuring that the truncation point is a valid UTF-8 character boundary.

Recommendation. Limit output using character iteration, `char_indices`, or a safe truncation helper, and do not perform unchecked raw-byte slicing of attacker-controlled UTF-8 strings on consensus-critical paths. Regression tests should cover multibyte UTF-8 input, boundary lengths, and oversized revert reasons.

Remediation and Review. Project records mark the issue as fixed, but the available record does not identify a corresponding pull request. The patch was not independently reviewed and the regression test was not re-run during this assessment; the finding is therefore recorded as Reported Fixed.

CFX-002 Non-pivot staking events can contaminate the PoS event cache

Severity	Impact	Likelihood	Status
Medium	High	Low	Reported Fixed

Affected Component	PoW / PoS staking-event bridge and event cache
Trigger Condition	Execution result from a non-local-pivot PoW fork enters the cache
Remediation Record	PR #3558
Code Location	crates/cfxcore/core/src/consensus/ consensus_inner/consensus_executor/ epoch_execution.rs; crates/cfxcore/core/src/pos/consensus/consensusdb/ mod.rs
Remediation Reference	PR #3558

Description. Execution results from a PoW fork that is not the local pivot may still write staking events into a height-indexed PoS cache. The read path validates only the hashes at the two ends of the requested range, allowing a fork event at an intermediate height to be returned as part of a canonical pivot-decision range.

Security Impact. PoS proposal generation or validation may consume staking events that do not belong to canonical PoW receipts, breaking consistency across the PoW/PoS event bridge and potentially affecting consensus availability or integrity.

Root Cause. The cache key is bound primarily to height rather than fully to the corresponding epoch/hash canonical identity, and the read path does not validate each event against the canonical pivot chain.

Recommendation. Gate cache writes on `on_local_pivot` and validate the canonical pivot-chain hash for each cached entry when reading. A more robust design is to key the cache by `(height, epoch_hash)`. Regression coverage should include non-pivot forks, pivot changes, and event reads spanning multiple heights.

Remediation and Review. Project records and PR #3558 indicate that the issue was fixed. The remediation regression tests were not re-run during this assessment, so the finding is recorded as Reported Fixed.

CFX-003 Unchecked GetTransactionsResponse hash count can cause an out-of-bounds panic

Severity	Impact	Likelihood	Status
Medium	Medium	Medium	Reported Fixed

Affected Component	Transaction synchronization protocol and sync-peer message handling
Attacker Capability	Peer able to establish a synchronization connection and return a malicious response
Remediation Record	PR #3509
Code Location	<code>crates/cfxcore/core/src/sync/request_manager/mod.rs :: request_transactions_from_tx_hashes();</code> <code>crates/cfxcore/core/src/sync/message/transactions.rs</code>
Remediation Reference	PR #3509

Description. A synchronization peer can return more `tx_hashes` than the number of requested `tx_hashes_indices`. Subsequent logic iterates over the number of response hashes and uses the same index to access the request-side index vector. If the response is longer than the request context, the code triggers a Rust bounds panic.

Security Impact. A malicious synchronization peer can send a decodable response with an invalid length relationship and cause the node message-processing path to panic, creating a process-level denial-of-service condition.

Root Cause. The protocol invariant between response length and the originating request context is not checked before indexed access is performed.

Recommendation. Explicitly validate `responded_tx_hashes.len() <= tx_hashes_indices.len()` in the handler or in `request_transactions_from_tx_hashes`. When the invariant fails, return a protocol error and handle the peer according to node policy rather than entering the indexing logic. Equivalent length and association invariants should be defined for other request/response message types.

Remediation and Review. Project records and PR #3509 indicate that the issue was fixed. Existing records describe regression coverage for both direct invocation and the complete handler path, but those tests were not re-run during this assessment; the finding is recorded as Reported Fixed.

CFX-004 BLOCKHASH(U256::MAX) can overflow without a guard

Severity	Impact	Likelihood	Status
Low	Medium	Low	Open

Affected Component	Legacy EVM BLOCKHASH Env path
Mainnet Reachability	Not reachable by new blocks on current mainnet; may affect unactivated chains, private/development networks, or historical replay
Code Location	crates/execution/executor/src/context.rs :: blockhash_from_env()

Description. The legacy Env path for EVM BLOCKHASH evaluates `number + 1` on a user-supplied block number. If the input is `U256::MAX`, the addition can panic in release builds where overflow checks are enabled. The State-backed path already guards `number > u64::MAX`, while the legacy Env path lacks an equivalent check.

Security Impact. On chains or historical execution environments that still use the legacy Env path, a contract can submit an extreme block number and panic a consensus execution worker. Existing records indicate that the relevant mainnet activation boundary was crossed on 2024-08-13 for Native Space and 2024-08-06 for eSpace, so new blocks on current mainnet do not reach this path.

Root Cause. The legacy Env path lacks the numeric range checks present in the State path and directly performs an addition that can overflow.

Recommendation. Add the `number > u64::MAX` guard to the Env path and use `checked_add`; return the zero hash for overflow or out-of-range values. Maintain boundary regression tests across Native Space, eSpace, and pre-/post-upgrade rule sets.

Remediation and Review. Existing records confirm that overflow checks are enabled in the release configuration and document the relevant mainnet activation boundaries. This assessment did not re-run dynamic validation on unactivated chains, private chains, development networks, or historical replay environments.

CFX-005 pre-CIP-645 BLOBHASH/BLOBBASEFEE metadata gap can panic

Severity	Impact	Likelihood	Status
Low	Medium	Low	Open

Affected Component	EVM opcode decoding / instruction metadata
Mainnet Reachability	Not reachable by new blocks on current mainnet; affects pre-CIP-645 execution environments
Code Location	<code>crates/execution/vm-interpreter/src/instructions.rs :: Instruction::info()</code>

Description. Opcodes 0x49 and 0x4a exist in the instruction enum, but the pre-CIP-645 instruction table lacks the corresponding metadata. In an execution environment where CIP-645 has not been activated, executing these opcodes can cause the metadata lookup to panic at `info()`.expect instead of returning an invalid-instruction error.

Security Impact. In a pre-CIP-645 execution environment, a single transaction containing either opcode may panic a consensus execution path. Current mainnet has already crossed the CIP-645 activation boundary, so new mainnet blocks no longer reach the legacy table; private chains, development networks, and historical replay may still be affected.

Root Cause. The opcode enum and the instruction metadata tables used for different hardfork states are not fully gated by a consistent activation invariant.

Recommendation. Explicitly gate 0x49 and 0x4a by CIP-645 during opcode decoding or metadata lookup and return `BadInstruction` before activation. Maintain an explicit test matrix covering every fork combination and instruction table.

Remediation and Review. Existing records include a `catch_unwind` compilation/run result and place mainnet CIP-645 activation at 2025-08-31. Whether an actual triggering transaction exists in the historical exposure window has not been confirmed.

CFX-006 eth_getLogs limits are disabled by default

Severity	Impact	Likelihood	Status
Low	Medium	Low	Open

Affected Component	EVM JSON-RPC eth_getLogs
Attacker Capability	Low-trust public RPC client
Reachability	Depends on whether the operator enables and publicly exposes EVM RPC
Code Location	crates/config/src/configuration.rs; crates/cfxcore/core/src/consensus/config.rs

Description. The EVM RPC configuration defaults both the log-query epoch range limit and the maximum result count to None. Limit-enforcement logic exists, but the defaults leave it disabled. A broad wildcard query may therefore scan and accumulate a large log set before the response-size limit becomes relevant.

Security Impact. When an operator exposes EVM RPC publicly, a low-trust client can issue a small number of expensive eth_getLogs requests that amplify CPU use, storage reads, memory pressure, and response latency. Archive nodes may be affected more severely.

Root Cause. Resource-limiting capabilities are present but are not enabled as a secure baseline for public RPC deployments, leaving request-cost control dependent on additional operator configuration.

Recommendation. Public RPC deployments should use conservative default limits for epoch range and result count, together with concurrency, timeout, request-body, and response-budget controls. Queries that exceed limits should fail early before large scans or result aggregation are performed. Archive, full-state, and public RPC deployments should use appropriately tiered resource policies.

Remediation and Review. Existing records confirm that the relevant configuration values default to None. Public RPC load testing was not re-run during this assessment.

CFX-007 eth_* / trace_* RPC methods bypass throttling coverage

Severity	Impact	Likelihood	Status
Low	Medium	Low	Open

Affected Component	eSpace / ETH JSON-RPC throttling
Attacker Capability	Low-trust public RPC client
Reachability	Depends on RPC exposure and runtime configuration
Code Location	crates/rpc/rpc-builder/src/eth/mod.rs; crates/config/src/configuration.rs; run/hydra.toml

Description. The throttling configuration loaded by the eSpace/ETH RPC server does not cover eth_*, trace_*, parity_*, and web3_* methods. With the default throttling_conf=None, no site-wide token bucket is enabled, and batch requests also lack an explicit count limit.

Security Impact. Trace, fee-history, and other expensive RPC operations lack the expected resource-isolation layer. An attacker can combine these methods with batch calls to amplify backend compute and I/O pressure and reduce availability of the public RPC service.

Root Cause. The throttling policy does not fully cover the actual externally exposed method set, and the default configuration does not provide a fallback global resource budget.

Recommendation. Include ETH, trace, parity, and web3 methods in a unified throttling policy and enable conservative limits by default. Set limits for batch count, request-body size, concurrency, timeout, and per-call resource budget. Expensive trace endpoints should use stricter limits than ordinary queries.

Remediation and Review. Existing records identify the coverage gap by statically comparing method names with the shipped throttling.toml. Throttling stress tests were not re-run during this assessment.

CFX-008 MCOPY does not zero-fill fully out-of-range source memory under legacy rules

Severity	Impact	Likelihood	Status
Low	Low	Medium	Open

Affected Component	EVM MCOPY / memory semantics
Mainnet Reachability	New blocks on current mainnet have limited exposure; a historical rule window exists
Code Location	crates/execution/vm-interpreter/src/interpreter/mod.rs :: copy_memory_to_memory()

Description. The underlying MCOPY memory-copy helper returns immediately when `source_offset >= mem_size`. Under legacy rules, the destination therefore retains its previous contents instead of being zero-filled as required by EIP-5656. CIP-645 indirectly expands memory through gas/memory accounting so that the early-return condition no longer occurs under current rules.

Security Impact. During the historical rule window, or on private chains configured with equivalent rules, Ethereum-derived contracts that rely on zero-fill semantics for fully out-of-range sources may silently observe stale destination memory. All Conflux nodes remain deterministic, so the issue is primarily an EVM compatibility and application-state risk rather than a current-mainnet consensus-split risk.

Root Cause. The low-level copy helper relies on side effects of upper-layer gas/memory expansion for semantic correctness rather than implementing complete zero-fill behavior itself.

Recommendation. Implement zero-fill explicitly in the copy helper so correctness does not depend on gasometer pre-expansion side effects. Add differential tests against geth for both fully and partially out-of-range sources.

Remediation and Review. Existing records identify a historical mainnet exposure window from 2024-08-13 to 2025-08-31. New blocks on current mainnet no longer trigger the semantic difference. Historical state impact and private-chain behavior should be assessed per deployment.

CFX-009 eSpace accepts unprotected pre-EIP-155 transactions

Severity	Impact	Likelihood	Status
Low	Medium	Low	Open

Affected Component	eSpace Ethereum legacy transaction validation
Attack Preconditions	Account, nonce, balance, gas, and target-state conditions must all be satisfied
Code Location	crates/primitives/src/transaction/ eth_transaction.rs; crates/cfxcore/core/src/verification.rs

Description. Ethereum legacy transactions with `v=27/28` decode to `chain_id=None`. The common validation logic checks the chain ID only when the value is `Some(chain_id)`, allowing an unprotected signature to enter the eSpace validation flow.

Security Impact. If account state, nonce, balance, gas, and target state all satisfy the required conditions, historical unprotected Ethereum transactions may theoretically be replayed across chains. The behavior may reflect a compatibility policy and does not automatically expire at a hardfork height.

Root Cause. The chain-ID validation path treats `None` as a compatibility branch that requires no chain binding, so pre-EIP-155 transactions do not receive domain isolation equivalent to EIP-155 transactions.

Recommendation. Define and document the compatibility policy for pre-EIP-155 transactions. If compatibility is not required, reject Ethereum-space transactions with `chain_id=None` in the transaction pool, execution, or consensus-related validation path. If compatibility must be retained, provide an explicit safety switch and document the risk for node operators.

Remediation and Review. Existing records confirm that `if let Some(chain_id)` causes `v=27/28` to bypass chain-binding validation. Practical replay still depends on account and chain state. No mainnet replay test was performed during this assessment.

CFX-010 CLI argument IDs can mismatch configuration fields and silently fail

Severity	Impact	Likelihood	Status
Low	Low	Low	Reported Fixed

Affected Component	CLI / runtime configuration mapping
Attack Surface	Local node configuration and operations
Remediation Record	PR #3564
Code Location	bins/conflux/src/cli.rs; crates/config/src/config_macro.rs
Remediation Reference	PR #3564

Description. The CLI accepts the documented `--port` option, while the generated configuration parser looks up hyphenated IDs derived from `RawConfiguration` field names, such as `tcp-port`. If the lookup error is silently ignored, `tcp_port` retains its default or TOML value rather than the value supplied on the command line.

Security Impact. Operators may believe a CLI setting is active while the node is actually running with a different value, potentially causing unintended network exposure, port conflicts, node unreachability, or divergence from expected runtime configuration. The issue is not a remote vulnerability, but it reduces configuration reliability and can indirectly weaken deployment security.

Root Cause. CLI option identifiers and runtime configuration fields do not have a stable explicit mapping, while lookup failures are silently ignored, allowing operator intent to diverge from actual runtime state.

Recommendation. Establish explicit mappings between CLI IDs and runtime configuration fields and retain compatibility mappings for legacy options, for example `--port -> tcp_port`. Expected configuration lookup failures should not be silently ignored. Add regression coverage that verifies parsing and consumption of every documented top-level CLI option.

Remediation and Review. Project records and PR #3564 indicate that the issue was fixed. CLI regression tests were not re-run during this assessment, so the finding is recorded as Reported Fixed.

7. Remediation Priorities

Priority	Finding	Action
P1	CFX-001	Independently review the High-severity remediation patch and execute UTF-8 boundary regression tests.
P2	CFX-002, CFX-003	Review both Medium-severity remediation patches and execute protocol/cache regression tests.
P3	CFX-006, CFX-007	Establish secure defaults for public RPC, including range, result count, throttling, batch, concurrency, request-body, and timeout limits.
P3	CFX-004, CFX-005, CFX-008	Remediate EVM edge cases that are not reachable or are trigger-limited on current mainnet to protect private chains, development networks, and historical replay environments.
P3	CFX-009	Define the pre-EIP-155 compatibility policy and decide whether to reject, retain, or gate the behavior behind an explicit safety option.

Priority	Finding	Action
P3	CFX-010	Independently review the CLI mapping remediation and associated regression coverage.

Remediation should first focus on all Reported Fixed High and Medium findings by independently reviewing the patches and performing the regression verification necessary to confirm that each fix is complete. The associated regression cases should then be retained as a security baseline for future releases. For the remaining Open Low findings, public RPC resource-boundary issues have the most direct operational relevance and should be hardened before historical EVM edge cases that are not reachable on current mainnet.

8. Limitations and Disclaimer

This report presents a repository-wide code security audit of `conflux-rust` at the target commit and is based on the code, configuration, and remediation information available during the review. The audit used a risk-oriented approach to systematically examine security-critical modules and high-risk paths, but it is not an exhaustive proof covering every line of code, every runtime environment, or every possible state combination.

This report should not be interpreted as having identified every potential security defect in the target code, nor as evidence that third-party dependencies, infrastructure, or operational environments outside the project code were assessed to the same depth. For issues recorded by the project as fixed, this engagement did not independently repeat dynamic regression testing for every patch. Different network configurations, private or development networks, and future code changes may also change the risk conclusions and applicability described here.

A security audit can materially reduce known and discoverable risk, but it cannot prove the absence of unknown vulnerabilities. Any subsequent material change affecting consensus, EVM execution, P2P protocols, RPC behavior, transaction validation, or node configuration should receive corresponding security review and regression testing.

Appendix A. Code Locations and Remediation References

This appendix summarizes the primary source locations and remediation references for each finding to support development, review, and regression testing in later releases. The listed source locations identify the principal trigger or remediation path and do not imply that analysis of a finding was limited to a single file.

Finding	Primary Code Location	Status / Remediation Reference
CFX-001	crates/execution/solidity-abi/src/ utils.rs :: string_revert_reason_decode()	Reported Fixed; no PR number provided
CFX-002	consensus_executor/ epoch_execution.rs; pos/consensus/consensusdb/mod.rs	Reported Fixed; PR #3558
CFX-003	sync/request_manager/mod.rs; sync/message/transactions.rs	Reported Fixed; PR #3509
CFX-004	crates/execution/executor/src/ context.rs :: blockhash_from_env()	Open
CFX-005	crates/execution/vm-interpreter/ src/instructions.rs :: Instruction::info()	Open
CFX-006	crates/config/src/configuration.rs; consensus/config.rs	Open
CFX-007	rpc-builder/src/eth/mod.rs; configuration.rs; run/hydra.toml	Open
CFX-008	vm-interpreter/src/interpreter/ mod.rs :: copy_memory_to_memory()	Open
CFX-009	primitives/src/transaction/ eth_transaction.rs; cfxc/core/src/verification.rs	Open
CFX-010	bins/conflux/src/cli.rs; crates/config/src/config_macro.rs	Reported Fixed; PR #3564